ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG





Báo cáo kỹ thuật

Phân tích ý kiến theo khía cạnh trên bộ dữ liệu OATS: từ pipeline sinh quadruple đến dashboard cho doanh nghiệp

Đề tài: Generative Aspect-Based Sentiment Analysis on OATS

Nhóm thực hiện:

Nguyễn Đức Mạnh	20235525
Hoàng Văn Nhân	20235542
Lê Nguyễn Phước Thành	20235561
Trần Quang Trọng	20235565

Hà Nội - Việt Nam

2026

Mục lục

Danh mục thuật ngữ và viết tắt **i**

1 Giới thiệu **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu 1

1.3 Phạm vi và đóng góp 2

1.3.1 Phạm vi 2

1.3.2 Đóng góp 2

1.4 Cấu trúc báo cáo 3

2 Cơ sở lý thuyết và khảo sát **4**

2.1 Bài toán ASQP 4

2.2 Bộ dữ liệu OATS 4

2.3 Hai phương pháp sinh được cài đặt 5

2.3.1 Paraphrase 5

2.3.2 GAS 6

2.4 Các hướng tiếp cận khác 6

2.5 Công nghệ sử dụng 7

3 Phân tích yêu cầu **8**

3.1 Tác nhân 8

3.2 Yêu cầu chức năng 9

3.3 Yêu cầu phi chức năng 9

4 Thiết kế hệ thống **11**

4.1 Kiến trúc tổng thể 11

4.2 Ba giao thức chia tập 12
4.3 Hợp đồng HTTP 12
4.4 Thiết kế dashboard theo khía cạnh 13

5 Triển khai **15**
5.1 Tầng dữ liệu 15
5.2 Dấu vân tay và chống lệch dữ liệu 15
5.3 Hai bộ chuyển đổi đầu ra 16
5.4 Chuẩn hóa và tính điểm 16
5.4.1 Chỉ số nới lỏng cho dashboard 16
5.5 Service dự đoán 17
5.6 Giao diện web 17

6 Kết quả và đánh giá **19**
6.1 Thiết lập thí nghiệm 19
6.2 Kết quả trên tập test 19
6.3 Vì sao không so sánh trực tiếp được với bài báo 20
6.4 Trạng thái các giao thức chia tập 21
6.5 Hạ tầng huấn luyện và giới hạn phần cứng 21
6.6 Kiểm thử phần mềm 23

7 Kết luận và hướng phát triển **24**
7.1 Kết luận 24
7.2 Hướng phát triển 24

Tài liệu tham khảo **27**

Tóm tắt nội dung

Một khách sạn nhận hàng nghìn nhận xét và thấy điểm trung bình giảm, nhưng điểm số đó không cho biết khách đang phàn nàn về điều gì. Bài toán *Aspect Sentiment Quad Prediction* (ASQP) tách mỗi ý kiến trong câu thành bộ bốn thành phần *(đối tượng, khía cạnh, cụm ý kiến, cảm xúc)*, nhờ đó có thể tổng hợp theo từng khía cạnh thay vì theo một điểm chung.

Báo cáo trình bày toàn bộ hệ thống nhóm đã xây dựng quanh bộ dữ liệu OATS [1]: một pipeline nghiên cứu (đọc dữ liệu, làm sạch, chia tập, huấn luyện, đánh giá, phân tích lỗi) cho hai mô hình sinh GAS [9] và Paraphrase [8] trên ba lĩnh vực; một service HTTP bọc checkpoint đã huấn luyện; và một giao diện web gồm hai phần: phân tích một câu, và dashboard nhận vào một tệp nhiều nhận xét rồi cho biết mỗi khía cạnh có bao nhiêu phần trăm nhận xét tiêu cực, kèm các cụm từ chê phổ biến.

Lượt chạy seed 42 trên ba lĩnh vực cho F1 khớp hoàn toàn trên tập test lần lượt là 0,227 / 0,111 / 0,347 với GAS và 0,210 / 0,144 / 0,389 với Paraphrase. Báo cáo chỉ ra một đặc điểm của bộ dữ liệu làm các con số này không so sánh trực tiếp được với bài báo: trong tập test có sẵn, 80,8% dòng của `coursera` và 86,7% dòng của `hotels` không có nhãn nào, trong khi tỷ lệ này ở tập train chỉ khoảng 25%. Phần kiểm thử gồm 150 test Python và 98 test frontend; luồng web đã được chạy thật đầu-cuối qua HTTP. Những phần chưa làm — chạy đa seed, chạy lại theo giao thức chia tập cấp bài đánh giá, và đo độ chính xác riêng cho cặp (khía cạnh, cảm xúc) mà dashboard dựa vào — được nêu rõ ở Chương 7 thay vì được suy đoán.

Danh mục thuật ngữ và viết tắt

Viết tắt	**Nghĩa**
ABSA	Aspect-Based Sentiment Analysis: phân tích cảm xúc theo khía cạnh
ASQP	Aspect Sentiment Quad Prediction: dự đoán bộ bốn (target, aspect, opinion, sentiment)
Quadruple	Bộ bốn thành phần của một ý kiến trong câu
Target	Đối tượng được nhắc tới; `NULL` nếu chỉ được ngầm hiểu
Aspect (category)	Khía cạnh, lấy từ danh mục cố định của từng lĩnh vực, ví dụ `rooms cleanliness`
Opinion	Cụm từ thể hiện ý kiến; `NULL` nếu ngầm hiểu
Sentiment	Một trong bốn nhãn: positive, negative, neutral, conflict
OATS	Opinion Aspect Target Sentiment, bộ dữ liệu của bài báo [1]
GAS	Generative ABSA, sinh chuỗi theo kiểu trích xuất [9]
Paraphrase	Sinh câu diễn giải tự nhiên cho mỗi quadruple [8]
T5	Text-to-Text Transfer Transformer, mô hình nền dùng cho cả hai phương pháp [4]
Exact match	Một dự đoán chỉ đúng khi cả bốn thành phần khớp chính xác
Leakage	Rò rỉ: các câu của cùng một bài đánh giá nằm ở nhiều tập khác nhau
Fingerprint	Mã băm SHA-256 của tệp dữ liệu, dùng để phát hiện dữ liệu đổi
Mock mode	Chế độ giao diện chạy bằng dữ liệu mẫu, không gọi mô hình

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Khi một doanh nghiệp nhìn vào điểm đánh giá trung bình, họ biết *có* vấn đề nhưng không biết vấn đề *nằm ở đâu*. Câu "Vị trí tuyệt vời, nhưng phòng bẩn và nhân viên thiếu nhiệt tình" vừa khen vừa chê; gộp thành một điểm sao sẽ mất toàn bộ thông tin hành động được. Phân tích cảm xúc ở mức câu cũng không đủ, vì một câu có thể chứa nhiều ý kiến trái chiều về nhiều đối tượng khác nhau.

ASQP giải quyết đúng chỗ thiếu đó: mỗi ý kiến được tách thành bộ bốn thành phần. Với câu trên, kết quả mong muốn là hai bộ: `(location, location general, "Great", positive)` và `(room, rooms cleanliness, "dirty", negative)`. Khi đã có các bộ bốn này cho hàng nghìn nhận xét, việc tổng hợp "khía cạnh nào bị chê nhiều nhất" trở thành phép đếm, và quan trọng hơn, có thể chỉ ra *vì sao* bị chê thông qua các cụm ý kiến đi kèm.

Bộ dữ liệu OATS [1] được công bố năm 2024 cho ba lĩnh vực: đánh giá thực phẩm trên Amazon, đánh giá khóa học trực tuyến Coursera và đánh giá khách sạn TripAdvisor. Điểm khiến OATS khó hơn các bộ ABSA trước đó là nó gán nhãn ở mức bài đánh giá nhiều câu, giữ cả các câu không mang ý kiến, và cho phép cả đối tượng lẫn cụm ý kiến ở dạng ngầm hiểu.

1.2 Mục tiêu

Nhóm đặt ba mục tiêu, theo thứ tự ưu tiên:

1. **Một pipeline nghiên cứu trung thực.** Chuẩn bị dữ liệu, huấn luyện, đánh giá và phân tích lỗi cho hai mô hình sinh trên ba lĩnh vực, trong đó mọi thao tác đều để lại dấu vết kiểm chứng được: nguồn dữ liệu, mã băm, cấu hình, danh mục khía cạnh.
2. **Kết quả thực nghiệm có điều kiện rõ ràng.** Báo cáo số liệu theo từng cặp (lĩnh vực, mô hình), không gộp ba lĩnh vực, không thay số của nhóm bằng số của bài báo, và nói rõ giao thức chia tập nào đã được dùng.
3. **Một sản phẩm dùng được.** Giao diện web cho phép phân tích một câu và, quan trọng hơn với doanh nghiệp, tải lên một tệp nhiều nhận xét để biết khía cạnh nào đang yếu.

1.3 Phạm vi và đóng góp

1.3.1 Phạm vi

Báo cáo giới hạn ở hai phương pháp sinh là GAS và Paraphrase trên nền T5-base, ba lĩnh vực của OATS và dữ liệu tiếng Anh. Các phương pháp khác được nhắc tới trong phần khảo sát để đối chiếu, không được cài đặt lại. Hệ thống phục vụ hiện dành cho chạy nội bộ: không có xác thực người dùng, không có giới hạn tần suất.

1.3.2 Đóng góp

- Pipeline đầy đủ với ba giao thức chia tập (`clean_split` theo bài đánh giá, `original_split` căn chỉnh với XML, `provided_split` dùng thẳng tệp chia sẵn) và cơ chế kiểm tra dấu vân tay dữ liệu xuyên suốt từ chuẩn bị dữ liệu đến đánh giá và so sánh.
- Lượt chạy seed 42 hoàn chỉnh: 6 lần huấn luyện, 6 lần đánh giá, 3 lần so sánh cùng lĩnh vực, 6 lần phân tích lỗi.
- Phát hiện và ghi nhận một đặc điểm dữ liệu ảnh hưởng trực tiếp tới việc diễn giải kết quả: tỷ lệ dòng không nhãn trong tập test chia sẵn của `coursera` và `hotels` cao bất thường (Mục 6.3).
- Service HTTP và giao diện web, trong đó có dashboard tổng hợp theo khía cạnh ở mức bài đánh giá, kèm quy tắc đếm được định nghĩa tường minh và công bố ngay

trên giao diện.

- Bộ kiểm thử tự động 150 test Python và 98 test frontend, cùng một lần kiểm chứng đầu-cuối chạy thật qua HTTP.

1.4 Cấu trúc báo cáo

Chương 2 trình bày cơ sở lý thuyết và khảo sát các phương pháp liên quan. Chương 3 phân tích yêu cầu. Chương 4 mô tả kiến trúc và các quyết định thiết kế. Chương 5 đi vào chi tiết triển khai. Chương 6 trình bày kết quả thực nghiệm, kiểm thử và các giới hạn đã xác định. Chương 7 kết luận và nêu hướng phát triển.

Chương 2

Cơ sở lý thuyết và khảo sát

2.1 Bài toán ASQP

Cho một câu s, mô hình phải sinh ra tập các bộ bốn

$$Q(s) = \{(t_i, a_i, o_i, p_i)\}_{i=1}^{n},$$

trong đó t_i là đối tượng, a_i là khía cạnh lấy từ danh mục cố định của lĩnh vực, o_i là cụm từ thể hiện ý kiến, và p_i là cảm xúc. Cả t_i và o_i có thể mang giá trị `NULL` khi chỉ được ngầm hiểu. Số bộ n có thể bằng 0 với câu không chứa ý kiến.

Chỉ số đánh giá là precision, recall và F1 theo *khớp hoàn toàn*: một dự đoán chỉ được tính đúng khi cả bốn thành phần trùng khớp. Đây là tiêu chí nghiêm ngặt — đoán đúng ba trên bốn thành phần vẫn bị tính sai hoàn toàn — và là lý do các con số F1 trong lĩnh vực này thấp hơn nhiều so với bài toán phân loại cảm xúc thông thường.

2.2 Bộ dữ liệu OATS

Bảng 2.1 thống kê dữ liệu thô do nhóm đo trực tiếp từ các tệp XML của bộ dữ liệu.

Bảng 2.1: Quy mô dữ liệu OATS, đo bằng chính bộ đọc XML của dự án.

Lĩnh vực	Bài đánh giá	Câu	Câu không nhãn	Số khía cạnh
`amazon_ff`	1.850	9.200	2.613 (28,4%)	13
`coursera`	1.850	8.985	2.825 (31,4%)	30
`hotels`	1.847	9.793	3.182 (32,5%)	37

Hai đặc điểm của bộ dữ liệu ảnh hưởng trực tiếp tới thiết kế hệ thống:

- **Phân bố nhãn lệch mạnh.** Khoảng 78–83% quadruple mang nhãn positive, 15–18% negative, chỉ 2–4% neutral, và nhãn thứ tư `conflict` xuất hiện rất hiếm (5–6 lần trên mỗi lĩnh vực). Hệ quả là mọi con số liên quan tới neutral và conflict đều dựa trên rất ít ví dụ.
- **Nhãn gắn ở mức bài đánh giá.** Một bài gồm nhiều câu; nếu chia tập ở mức câu thì các câu của cùng một bài sẽ nằm ở cả tập train lẫn test. Dự án vì vậy coi *không rò rỉ ở mức bài đánh giá* là một ràng buộc cần được kiểm tra bằng mã, không phải một giả định.

2.3 Hai phương pháp sinh được cài đặt

2.3.1 Paraphrase

Phương pháp Paraphrase [8] biến mỗi quadruple thành một câu tiếng Anh tự nhiên theo khuôn "*aspect* is *phrase* because *target* is *opinion*", trong đó cảm xúc được ánh xạ thành một tính từ: positive → `great`, negative → `bad`, neutral → `ok`. Nhiều quadruple trong một câu được nối bằng dấu phân cách `[SSEP]`. Đối tượng hoặc ý kiến ngầm hiểu được viết là `it`. Ưu điểm là chuỗi đích gần với phân bố ngôn ngữ mà T5 đã học trong giai đoạn tiền huấn luyện.

OATS có nhãn thứ tư là `conflict`, không nằm trong ba tính từ của bài báo gốc. Dự án mở rộng bằng từ `mixed` và ghi rõ đây là lựa chọn của nhóm, không phải thiết lập của bài báo.

2.3.2 GAS

Bài báo GAS [9] trình bày hai kiểu sinh cho các bài toán ABSA, trong đó có kiểu trích xuất. Bài báo OATS áp dụng GAS cho ASQP nhưng *không công bố* cú pháp chuỗi đích cụ thể. Dự án vì vậy chọn một biểu diễn trích xuất có cấu trúc: mảng JSON gồm các bộ bốn chuỗi theo thứ tự cố định.

Ranh giới cần giữ

Vì cú pháp đích của GAS-T5 trong bài báo OATS không được công bố, kết quả GAS trong báo cáo này là của *một cách triển khai theo hướng trích xuất do nhóm chọn*, không phải tái tạo chính xác thí nghiệm của bài báo. Mọi so sánh với cột GAS-T5 trong bài báo phải kèm theo lưu ý này.

2.4 Các hướng tiếp cận khác

Bài báo OATS còn báo cáo Template-ILO và Template-DLO [2] — tăng cường dữ liệu bằng cách hoán vị thứ tự các thành phần trong chuỗi đích — và GEN-SCL-NAT [3] — kết hợp học đối lập có giám sát với một cấu trúc đích giàu thông tin hơn. Cả ba đều cùng họ "sinh chuỗi bằng T5", khác nhau chủ yếu ở dạng chuỗi đích và cách tăng cường dữ liệu. Nhóm không cài đặt lại các phương pháp này; chúng được dùng làm mốc đối chiếu về mặt số liệu.

Ở phía sản phẩm, các công cụ phân tích nhận xét thương mại thường dừng ở mức gán nhãn cảm xúc cho từng chủ đề được phát hiện tự động. Khoảng trống mà hệ thống này nhắm tới là giữ nguyên *cụm từ ý kiến* do mô hình trích ra, nhờ đó dashboard trả lời được câu hỏi "vì sao khía cạnh này bị chê" chứ không chỉ "khía cạnh này bị chê".

2.5 Công nghệ sử dụng

Bảng 2.2: Công nghệ chính và vai trò.

Thành phần	**Công nghệ**
Pipeline nghiên cứu	Python 3.12, PyTorch, Transformers [7], mô hình nền T5-base [4]
Service dự đoán	FastAPI [5] + Uvicorn, nạp checkpoint qua chính mã của pipeline
Giao diện web	React 19, TypeScript, Vite
Kiểm thử	pytest (150 test), Vitest + Testing Library (98 test)

Chương 3

Phân tích yêu cầu

3.1 Tác nhân

- **Người nghiên cứu** chạy pipeline, cần kết quả tái lập được và bằng chứng về dữ liệu đã dùng.
- **Người phân tích của doanh nghiệp** có một tệp nhận xét và cần biết khía cạnh nào đang yếu, vì sao, và có đủ dữ liệu để kết luận hay không.
- **Người phát triển** nối giao diện với mô hình, cần một giao diện lập trình ổn định và chạy được cả khi chưa có checkpoint.

3.2 Yêu cầu chức năng

Bảng 3.1: Yêu cầu chức năng chính.

Mã	Tác nhân	Mô tả
CN1	Nghiên cứu	Chuẩn bị dữ liệu từ nguồn thô, ghi lại nguồn, mã băm và danh mục khía cạnh
CN2	Nghiên cứu	Huấn luyện một thí nghiệm theo tệp cấu hình, chọn checkpoint tốt nhất theo F1 tập dev
CN3	Nghiên cứu	Đánh giá trên tập test, xuất metrics, dự đoán và các ca lỗi
CN4	Nghiên cứu	So sánh hai mô hình trong cùng một lĩnh vực, có kiểm tra cùng dữ liệu
CN5	Phát triển	Dự đoán một câu qua HTTP
CN6	Phát triển	Dự đoán nhiều câu trong một lần gọi, phục vụ xử lý theo lô
CN7	Phân tích	Phân tích một câu trên giao diện và xem đủ bốn thành phần
CN8	Phân tích	Tải lên tệp nhận xét (CSV/TXT/JSONL/JSON) và xem tỷ lệ cảm xúc theo từng khía cạnh
CN9	Phân tích	Xem các cụm từ chê phổ biến và các nhận xét cụ thể của một khía cạnh
CN10	Phân tích	Xuất bảng tổng hợp ra CSV

3.3 Yêu cầu phi chức năng

- **Trung thực về bằng chứng.** Giao diện không được hiển thị con số chất lượng mô hình khi chưa có bằng chứng; chế độ dữ liệu mẫu phải được dán nhãn rõ ràng.
- **Không rò rỉ chi tiết nội bộ.** Lỗi phía máy chủ chỉ hiện thông báo ngắn và mã lỗi; vết lỗi nằm trong log.
- **Chạy được khi chưa có mô hình.** Thiếu checkpoint phải cho ra trạng thái `MODEL_NOT_READY` có thể hiển thị, không phải sự cố.
- **Tiếp cận được.** Nhãn gắn đúng ô nhập, thông báo bằng vùng `aria-live`, cảm xúc phân biệt bằng chữ và ký hiệu chứ không chỉ bằng màu [6].

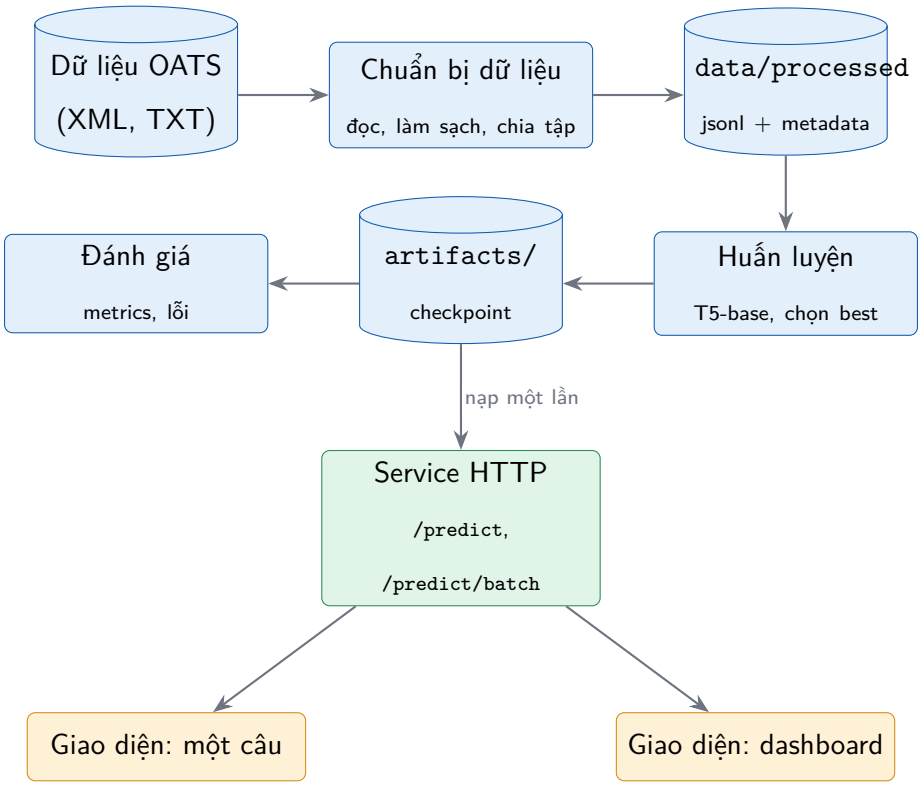
- **Tái lập được.** Mỗi thí nghiệm ghi lại cấu hình, dấu vân tay dữ liệu và commit mã nguồn.

Chương 4

Thiết kế hệ thống

4.1 Kiến trúc tổng thể

Hệ thống gồm ba tầng tách biệt, nối với nhau bằng những ranh giới hẹp và được định nghĩa rõ: tầng nghiên cứu sinh ra checkpoint, tầng phục vụ bọc checkpoint thành HTTP, tầng giao diện tiêu thụ HTTP (Hình 4.1).



Hình 4.1: Ba tầng của hệ thống. Tầng nghiên cứu và tầng sản phẩm chỉ gặp nhau tại thư mục checkpoint và tại hợp đồng HTTP.

Lý do tách như vậy: tầng nghiên cứu phụ thuộc PyTorch và dữ liệu nặng, trong khi

giao diện cần chạy được trên máy không có GPU. Mã nạp mô hình được nhập khẩu *bên trong hàm*, nên toàn bộ phần xử lý dữ liệu và đánh giá vẫn chạy được khi máy không cài PyTorch — đây cũng là điều cho phép bộ kiểm thử chạy nhanh trong môi trường phát triển.

4.2 Ba giao thức chia tập

Bảng 4.1: So sánh ba giao thức chia tập mà pipeline hỗ trợ.

Giao thức	**Cách làm và tính chất**	**Kiểm tra rò rỉ**
`clean_split`	Đọc XML, chia *theo bài đánh giá* với seed cố định	Có, bằng mã
`original_split`	Dùng tệp chia sẵn, căn chỉnh từng dòng với câu trong XML để lấy lại định danh bài	Có, nếu căn chỉnh thành công
`provided_split`	Đọc thẳng tệp `train/dev/test.txt`, mỗi dòng là một mẫu	Không thể: TXT không có định danh bài

Lượt chạy seed 42 dùng `provided_split`. Lựa chọn này giữ đúng phân chia của tác giả bộ dữ liệu, nhưng đánh đổi: bảo đảm "không rò rỉ ở mức bài đánh giá" không còn kiểm chứng được bằng mã. Hệ thống không che giấu đánh đổi đó — metadata ghi `review_level_leakage_check.status = not_verifiable`, và phần audit từ chối phát biểu `no_review_leakage` cho dữ liệu dạng dòng.

4.3 Hợp đồng HTTP

Giao diện và mô hình chỉ gặp nhau qua hai điểm cuối. Dạng dữ liệu được chọn trùng với dạng mà `capstone.predict` đã sinh ra, nên tầng phục vụ chỉ là lớp bọc mỏng chứ không phải một cách biểu diễn thứ hai.

```
POST /predict
{"text": "The room was dirty.", "domain": "hotels", "model_type": "paraphrase"}

200 OK
{"text": ..., "domain": ..., "model_type": ...,
 "quadruples": [{"target": "room", "aspect": "rooms cleanliness",
                 "opinion": "dirty", "sentiment": "negative"}],
 "raw_generation": "...", "invalid_predictions": []}

POST /predict/batch
{"domain": "hotels", "model_type": "paraphrase",
 "items": [{"id": "row-2", "text": "..."}, {"id": "row-3", "text": "..."}]}

200 OK  {"results": [{"id": "row-2", "quadruples": [...]}, ...]}

Lỗi (mọi điểm cuối)
{"error": {"code": "MODEL_NOT_READY", "message": "..."}}
```

Ba quyết định đáng nói:

- **Định danh do phía gọi đặt.** Mỗi phần tử trong lô mang `id` riêng và máy chủ trả lại đúng định danh đó. Phía giao diện kiểm tra tập định danh trả về khớp tập đã gửi, nên kết quả không thể gắn nhầm sang nhận xét khác khi thứ tự thay đổi.
- **Một dạng lỗi duy nhất.** Mọi lỗi, kể cả lỗi kiểm tra dữ liệu vào, đều mang dạng `{error:{code, message}}`; giao diện chỉ cần một nhánh xử lý.
- **Thiếu mô hình là một trạng thái, không phải sự cố.** Cặp (lĩnh vực, mô hình) chưa khai báo hoặc thiếu checkpoint trả về `MODEL_NOT_READY` kèm mã HTTP 503.

4.4 Thiết kế dashboard theo khía cạnh

Phần này trả lời câu hỏi của doanh nghiệp: *khía cạnh nào đang kéo đánh giá xuống?* Thiết kế xoay quanh một quyết định duy nhất nhưng quan trọng: **đơn vị đếm là bài đánh giá, không phải lượt nhắc**.

Nếu đếm theo lượt nhắc, một bài chê phòng bẩn ba lần sẽ được tính ba lần và làm lệch tỷ lệ. Vì vậy mỗi bài đánh giá chỉ đóng góp tối đa một lần cho mỗi khía cạnh, theo quy tắc gộp ở Bảng 4.2.

Bảng 4.2: Quy tắc gộp nhiều lượt nhắc cùng một khía cạnh trong cùng một bài đánh giá.

Nhãn bài	**Điều kiện**
Conflict	Có nhãn `conflict`, hoặc vừa có lượt tích cực vừa có lượt tiêu cực
Negative	Có ít nhất một lượt tiêu cực và không có lượt tích cực
Positive	Có ít nhất một lượt tích cực và không có lượt tiêu cực
Neutral	Chỉ có các lượt trung tính

Quy tắc này khiến các tỷ lệ của một khía cạnh luôn cộng lại bằng 100%, và giữ nhãn `conflict` tách riêng thay vì gộp vào một nhóm khác — phù hợp với việc OATS coi conflict là một nhãn độc lập. Quy tắc được in ngay trên giao diện trong mục "How are reviews counted?" để người đọc bảng không phải đoán.

Ba lựa chọn thiết kế còn lại đều nhằm chống kết luận sai:

- **Ngưỡng số mẫu tối thiểu** (mặc định 3 bài đánh giá) ẩn những khía cạnh quá ít dữ liệu, tránh hiển thị "100% tiêu cực" dựa trên một nhận xét duy nhất.
- **Hai mức gom nhóm:** theo danh mục đầy đủ (`rooms cleanliness`) hoặc theo nhóm lớn (`rooms`), cho phép nhìn tổng quan trước rồi mới đi sâu.
- **Luôn kèm bằng chứng:** mỗi khía cạnh mở ra được danh sách cụm từ chê phổ biến và các nhận xét gốc, nên người dùng kiểm tra lại được thay vì tin vào một con số.

Chương 5

Triển khai

5.1 Tầng dữ liệu

Bộ đọc XML kiểm tra tính nhất quán của chú thích ngay khi đọc: với mỗi quadruple, nếu dữ liệu cung cấp vị trí ký tự thì vị trí đó phải trỏ đúng vào cụm từ tương ứng trong câu. Các bất thường không làm dừng chương trình mà được ghi thành danh sách `DataIssue` có lý do, kèm đường dẫn nguồn và số dòng, rồi tổng hợp vào metadata.

Quyết định quan trọng ở bước làm sạch: **bài đánh giá không có nhãn được giữ lại**. Không có nhãn có thể nghĩa là "câu này thật sự không chứa ý kiến" hoặc "người gán nhãn bỏ sót"; dữ liệu không phân biệt được hai trường hợp đó. Vì vậy pipeline coi đây là trạng thái chưa rõ và chỉ cho phép loại bỏ thông qua một tệp kê khai có dẫn chứng; nếu tệp kê khai chứa định danh không tồn tại, bước chuẩn bị dữ liệu dừng lại thay vì im lặng bỏ qua.

5.2 Dấu vân tay và chống lệch dữ liệu

Đây là cơ chế giữ cho kết quả không bị "trôi". Bước chuẩn bị dữ liệu ghi vào `metadata.json`: mã băm SHA-256 của từng tệp nguồn và từng tệp tập dữ liệu, cấu hình làm sạch, danh mục khía cạnh của lĩnh vực và, với `clean_split`, cả seed. Trước khi huấn luyện, trình huấn luyện đối chiếu metadata này với cấu hình thí nghiệm; khi đánh giá, bước đánh giá kiểm tra lại dấu vân tay; khi so sánh hai mô hình, chương trình yêu cầu hai thí nghiệm có *cùng dấu vân tay tập test và cùng danh mục khía cạnh*, chứ không chỉ cùng số lượng mẫu.

Trong lần rà soát trước khi gộp mã, nhóm phát hiện metadata của giao thức `provided_split` lưu đường dẫn tuyệt đối của máy chạy. Hệ quả là dữ liệu đã xử lý không dùng lại được trên máy khác, vì bước kiểm tra sẽ không tìm thấy tệp nguồn. Lỗi đã được sửa: đường dẫn nay được lưu đúng như cấu hình, thường là đường dẫn tương đối tới gốc dự án.

5.3 Hai bộ chuyển đổi đầu ra

Mỗi mô hình có một cặp hàm: một hàm biến danh sách quadruple thành chuỗi đích để huấn luyện, và một hàm đọc ngược chuỗi sinh ra thành quadruple. Hàm đọc ngược phải chịu được mọi thứ mô hình có thể sinh ra, kể cả chuỗi hỏng, nên nó không ném lỗi mà trả về hai danh sách: các quadruple hợp lệ, và các mẩu không đọc được kèm lý do. Các mẩu hỏng này đi thẳng vào kết quả đánh giá dưới dạng `invalid_predictions`, nhờ đó "mô hình sinh sai cú pháp" là một đại lượng đo được chứ không phải thông tin bị mất.

5.4 Chuẩn hóa và tính điểm

Trước khi so khớp, cả nhãn chuẩn lẫn dự đoán đều đi qua bước chuẩn hóa: gộp khoảng trắng thừa, nhận diện các cách viết của giá trị ngầm hiểu, và đối chiếu khía cạnh với danh mục của lĩnh vực. Việc đối chiếu danh mục quan trọng vì mô hình sinh tự do có thể tạo ra một khía cạnh không tồn tại; những trường hợp đó được đánh dấu thay vì âm thầm tính là sai.

5.4.1 Chỉ số nới lỏng cho dashboard

Chỉ số chính của bài toán đòi đúng cả bốn thành phần, trong đó có hai chuỗi tự do là đối tượng và cụm ý kiến. Nhưng dashboard ở Mục 4.4 chỉ tiêu thụ cặp *(khía cạnh, cảm xúc)*. Một dự đoán `(bathroom, rooms cleanliness, "filthy", negative)` cho nhãn chuẩn `(room, rooms cleanliness, "dirty", negative)` bị chỉ số chính tính là *sai hoàn toàn*, trong khi với dashboard nó *đúng*: vẫn là một bài đánh giá tiêu cực về độ sạch phòng.

Nếu chỉ có chỉ số chính thì không có cách nào biết dashboard đáng tin đến đâu. Vì vậy bước đánh giá xuất thêm hai chỉ số nới lỏng, tính bằng đúng cơ chế đếm chống trùng lặp của chỉ số chính nhưng sau khi chiếu quadruple xuống một tập con các trường: chỉ `aspect`, và cặp `aspect + sentiment`. Trường `definition` đi kèm trong tệp JSON ghi

thẳng rằng đây là chỉ số nới lỏng và không được trình bày như kết quả ASQP — để con số dễ nhìn hơn không bị trích dẫn nhầm chỗ.

Ngoài P/R/F1 khớp hoàn toàn, bước đánh giá còn xuất các chẩn đoán: độ chính xác theo từng thành phần, kết quả theo số lượng quadruple trong câu, và phân loại lỗi (thiếu quadruple, thừa quadruple, sai một thành phần, hỏng về đối tượng hoặc ý kiến ngầm hiểu).

Một lỗi đếm trùng đã được sửa

Khi rà soát, nhóm phát hiện một quadruple đã được báo cáo trong danh sách "sai một thành phần" vẫn bị đếm thêm một lần nữa vào danh sách "hỏng ý kiến ngầm hiểu". Nhánh xử lý đối tượng đã được khử trùng lặp trước đó, nhánh ý kiến thì chưa. Lỗi này chỉ ảnh hưởng *số liệu chẩn đoán*, không ảnh hưởng P/R/F1, và đã được sửa kèm ba test kiểm chứng thất bại trên mã cũ. Con số `implicit_opinion_failure` công bố trong báo cáo lượt chạy seed 42 được tính trước bản sửa này, nên không so sánh trực tiếp được với con số tương ứng của đối tượng.

5.5 Service dự đoán

Service có hai phần. Phần đăng ký ánh xạ cặp (lĩnh vực, mô hình) tới một thư mục checkpoint theo tệp cấu hình, nạp mô hình *lười* ở lần gọi đầu tiên rồi giữ lại trong bộ nhớ. Phần ứng dụng chuyển hai điểm cuối thành lời gọi tới mô hình đã nạp.

Quyết định đáng nói nhất là cách xử lý thất bại: mọi lỗi khi nạp mô hình — thiếu thư viện, thiếu tệp, checkpoint hỏng — đều được ghi log đầy đủ rồi chuyển thành một `MODEL_NOT_READY` duy nhất gửi cho phía gọi, và kết quả thất bại đó được nhớ lại để không thử nạp lại ở từng yêu cầu. Nhờ vậy một checkpoint hỏng không biến thành hàng nghìn lần nạp mô hình thất bại.

5.6 Giao diện web

Giao diện tách thành ba lớp: các thành phần hiển thị, một giao diện lập trình `PredictionService`, và hai hiện thực của giao diện đó — một gọi HTTP thật, một chạy dữ liệu mẫu ngay trong trình duyệt. Thành phần hiển thị không biết hiện thực nào đang chạy; chuyển đổi bằng biến môi trường, không phải bằng sửa mã.

Chế độ dữ liệu mẫu không phải là mã tạm thời. Nó cho phép phát triển và kiểm thử toàn bộ giao diện khi chưa có checkpoint, và được dán nhãn rõ trên màn hình để không ai nhầm dữ liệu mẫu với dự đoán thật.

Hai chi tiết kỹ thuật đáng chú ý:

- **Chống ghi đè kết quả cũ.** Mỗi yêu cầu mới hủy yêu cầu đang chạy; phản hồi đến muộn của yêu cầu đã bị thay thế sẽ bị bỏ qua, nên kết quả hiển thị luôn ứng với lần bấm cuối cùng.
- **Xử lý tệp theo lô, có thể tiếp tục.** Tệp nhiều nhận xét được chia thành các lô 20 dòng. Các lô đã xong được giữ lại, nên khi gặp lỗi mạng giữa chừng, nút "Resume" chỉ gửi lại phần còn thiếu thay vì làm lại từ đầu.

Chương 6

Kết quả và đánh giá

6.1 Thiết lập thí nghiệm

Sáu thí nghiệm (2 mô hình $\times$ 3 lĩnh vực) dùng chung thiết lập: T5-base, 20 epoch, kích thước lô 16, tốc độ học $3 \cdot 10^{-4}$, độ dài chuỗi vào/ra 128, giải mã tham lam, seed 42, giao thức `provided_split`. Checkpoint được chọn theo F1 trên tập dev. Mọi số liệu dưới đây là của *một* seed duy nhất.

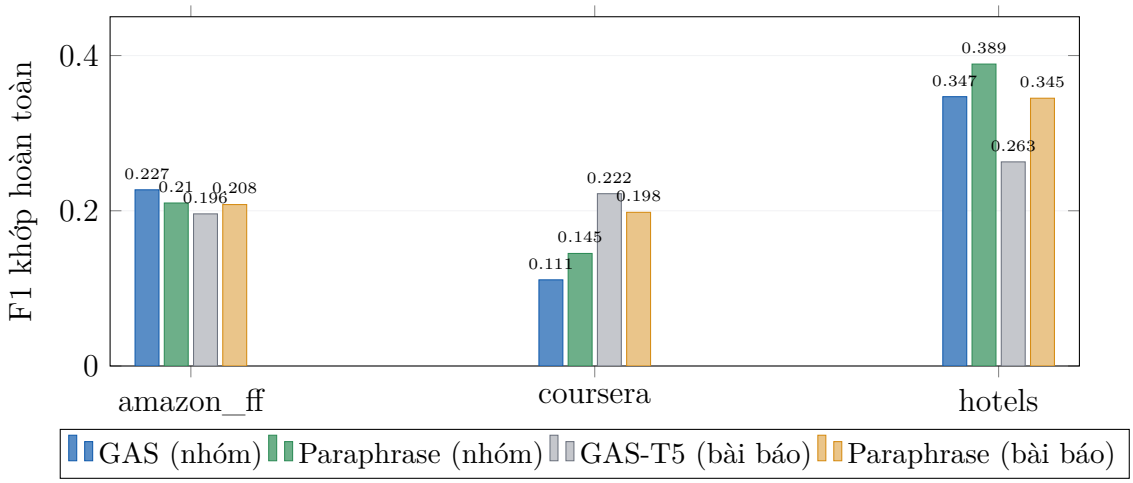
6.2 Kết quả trên tập test

Bảng 6.1: Kết quả seed 42 trên tập test, khớp hoàn toàn cả bốn thành phần.

Lĩnh vực	Mô hình	Epoch	Dev F1	P	R	F1	TP	FP
`amazon_ff`	GAS	14	0,297	0,247	0,209	**0,227**	202	615
`coursera`	GAS	5	0,295	0,071	0,254	**0,111**	60	785
`hotels`	GAS	19	0,442	0,299	0,413	**0,347**	100	234
`amazon_ff`	Paraphrase	5	0,294	0,215	0,205	**0,210**	198	721
`coursera`	Paraphrase	3	0,308	0,095	0,309	**0,145**	73	699
`hotels`	Paraphrase	16	0,454	0,337	0,459	**0,389**	111	218

So sánh hai mô hình trong cùng lĩnh vực: Paraphrase cao hơn GAS ở `coursera` (+0,034) và `hotels` (+0,042), thấp hơn ở `amazon_ff` ($-$0,017). Ba quan sát từ một seed duy nhất không đủ để kết luận mô hình nào tốt hơn nói chung; muốn phát biểu như vậy cần giao

thức năm seed như bài báo.



Hình 6.1: Kết quả của nhóm so với số công bố trong bài báo OATS. Hai nhóm cột *không* cùng điều kiện thí nghiệm: số của nhóm là một seed theo `provided_split`, số của bài báo là trung bình năm seed; cú pháp đích của GAS cũng khác. Hình chỉ để định vị độ lớn, không phải một phép so sánh công bằng.

6.3 Vì sao không so sánh trực tiếp được với bài báo

Khi rà soát kết quả, nhóm nhận thấy `coursera` có 785 dự đoán sai dương so với chỉ 60 dự đoán đúng — một tỷ lệ bất thường. Nguyên nhân nằm ở dữ liệu chứ không ở mô hình. Bảng 6.2 cho thấy tỷ lệ dòng không mang nhãn nào trong các tệp chia sẵn.

Bảng 6.2: Tỷ lệ dòng không có nhãn trong các tệp chia sẵn, do nhóm đếm trực tiếp.

Lĩnh vực	Train	Dev	Test
`amazon_ff`	29,3%	23,5%	25,8%
`coursera`	26,2%	23,9%	**80,8%**
`hotels`	24,5%	39,4%	**86,7%**

Tổng số dòng và tổng số dòng không nhãn của ba tập cộng lại khớp với dữ liệu XML, nên đây đúng là phân chia gốc của bộ dữ liệu chứ không phải lỗi xử lý. Nhưng hệ quả rất lớn: ở `hotels`, hơn 86% câu trong tập test không có nhãn nào, nên mỗi quadruple mà mô hình sinh ra cho những câu đó đều bị tính là sai dương, kéo precision xuống. Tỷ lệ này ở tập train chỉ khoảng 25%, tức là phân bố giữa lúc học và lúc chấm rất khác nhau.

Kết luận cần giữ đúng mức

Các con số trong Bảng 6.1 là kết quả hợp lệ *trong giao thức* `provided_split`. Chúng không tương đương với số của bài báo, và chênh lệch giữa các lĩnh vực phản ánh đặc điểm phân chia dữ liệu ít nhất ngang với việc phản ánh chất lượng mô hình.

6.4 Trạng thái các giao thức chia tập

Khi kiểm chứng lại, nhóm chạy thử cả ba giao thức trên dữ liệu thật:

Bảng 6.3: Kết quả chạy thử bước chuẩn bị dữ liệu theo từng giao thức.

Giao thức	`amazon_ff`	`coursera`	`hotels`
`clean_split`	Thành công	Thành công	Thành công
`provided_split`	Thành công	Thành công	Thành công
`original_split`	Lỗi (dòng 12)	Lỗi (dòng 11)	Lỗi (dòng 48)

Lỗi của `original_split` là "một dòng khớp với nhiều câu XML chưa được gán và nhãn không phân biệt được duy nhất một câu": khi cùng một câu xuất hiện nhiều lần trong bộ dữ liệu, bước căn chỉnh không biết dòng đó thuộc về bài đánh giá nào. Phát hiện quan trọng là `clean_split` — giao thức có kiểm tra rò rỉ — vẫn chạy được trên cả ba lĩnh vực. Nói cách khác, giao thức chính của dự án không hỏng, và việc chạy lại ma trận thí nghiệm theo `clean_split` là khả thi. Repository giữ sẵn sáu cấu hình `clean_split` với thư mục dữ liệu và thư mục kết quả riêng để giao thức này không bị bỏ quên.

Dữ liệu cho giao thức này đã được chuẩn bị xong cho cả ba lĩnh vực, chia 80/10/10 ở mức bài đánh giá, và bước audit xác nhận `no_review_leakage: true` ở cả ba. Phần còn thiếu chỉ là huấn luyện, vì lý do phần cứng ở Mục 6.5.

6.5 Hạ tầng huấn luyện và giới hạn phần cứng

Nhóm đã thử chạy ma trận `clean_split` trên một máy xách tay có GPU RTX 5050 8 GB VRAM và 16 GB RAM. Lần thử không hoàn tất, và các số đo thu được đáng ghi lại vì chúng quyết định việc chọn máy cho lần chạy sau.

Bảng 6.4: Thời gian một bước huấn luyện, đo trên RTX 5050 8 GB, batch 16, chuỗi vào/ra 128.

Mô hình	**Precision**	**Giây/bước**
Paraphrase	fp32	0,5
GAS	fp32	3,8
GAS	bf16	29,0

Ba điều rút ra:

- **Nút thắt là VRAM, không phải độ dài chuỗi.** GAS chậm gấp 8 lần Paraphrase trên cùng mô hình và cùng kích thước lô. Mức chiếm VRAM đo được là 7,4–7,6 GB trên card 8 GB; phần vượt bị hệ điều hành đẩy sang RAM hệ thống, và tốc độ sụp theo.
- **Mixed precision không phải lúc nào cũng nhanh hơn.** Giả thiết ban đầu của nhóm là bật bf16 sẽ giảm bộ nhớ và tăng tốc. Đo thực tế cho kết quả ngược lại: bf16 chậm hơn fp32 tám lần, vì autocast vẫn giữ trọng số gốc ở fp32 rồi tạo thêm bản bf16 cho từng phép toán, tức *tăng* đỉnh bộ nhớ trên một card vốn đã đầy. Tuỳ chọn `training.precision` vẫn được giữ lại trong mã, mặc định `fp32`, kèm cảnh báo này.
- **Hai lượt huấn luyện không được chạy song song.** Một lần chạy chồng nhau làm lượt đầu dừng với `CUDA error: CUBLAS_STATUS_EXECUTION_FAILED` ở epoch 12 sau hơn 45 phút. Pipeline không hỗ trợ chạy tiếp dở dang, nên toàn bộ lượt đó phải làm lại.

Các bài học này được ghi thành hướng dẫn vận hành trong repository để lần chạy sau trên máy mạnh hơn không lặp lại, trong đó có một bước bắt buộc: đo giây/bước trong vài phút đầu và dừng lại hỏi nếu tốc độ cho thấy tổng thời gian vượt dự tính.

6.6 Kiểm thử phần mềm

Bảng 6.5: Bộ kiểm thử tự động.

Phạm vi	Số test	Nội dung chính
Pipeline (pytest)	133	Đọc XML, làm sạch, chia tập và chống rò rỉ, dấu vân tay, hai bộ chuyển đổi đầu ra, chuẩn hóa và tính điểm (gồm chỉ số nới lỏng), đường dẫn cấu hình lồng thư mục, lựa chọn precision, các lệnh dòng lệnh
Service (pytest)	27	Hợp đồng hai điểm cuối, echo định danh theo lô, nạp mô hình một lần, các đường lỗi `MODEL_NOT_READY` và `PREDICTION_FAILED`, CORS, đọc tệp cấu hình phục vụ
Giao diện (Vitest)	98	Trạng thái tải/lỗi/rỗng, hiển thị quadruple và giá trị ngầm hiểu, đọc tệp CSV/TXT/JSONL/JSON, quy tắc đếm theo bài đánh giá, xuất CSV, chia lô và tiếp tục khi lỗi

Ngoài test tự động, luồng web đã được chạy thật đầu-cuối: service chạy trên cổng 8000, giao diện chạy ở chế độ API và gọi qua proxy, thao tác thật trên trình duyệt cho thấy nhãn chế độ đổi thành "API mode", phân tích một câu trả về quadruple đúng, cặp lĩnh vực chưa khai báo hiện `MODEL_NOT_READY` kèm nút thử lại, và tệp mẫu 30 nhận xét đi qua điểm cuối theo lô rồi lên dashboard.

Điều kiện của lần kiểm chứng đầu-cuối

Máy phát triển không cài PyTorch và không có checkpoint, nên lần chạy đầu-cuối dùng một bộ dự đoán giả đặt *ngoài* mã sản phẩm. Phép thử này xác nhận hợp đồng HTTP, việc chia lô và toàn bộ luồng giao diện; nó *không* xác nhận chất lượng dự đoán của mô hình thật.

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận

Hệ thống đã đi trọn một vòng: từ dữ liệu OATS thô, qua pipeline có kiểm chứng, tới sáu mô hình đã huấn luyện, và tới một giao diện trả lời được câu hỏi kinh doanh "khía cạnh nào đang yếu và vì sao". Phần nhóm cho là có giá trị nhất không phải các con số F1 — vốn còn thấp và chỉ từ một seed — mà là việc mọi phát biểu trong hệ thống đều gắn với bằng chứng: dấu vân tay dữ liệu chặn việc đánh giá nhầm tập, trạng thái không kiểm chứng được thì được ghi là không kiểm chứng được, và chế độ dữ liệu mẫu được dán nhãn thay vì giả làm dự đoán thật.

7.2 Hướng phát triển

1. **Lấy số cho cặp (khía cạnh, cảm xúc).** Chỉ số đã được cài đặt (Mục 5.4.1) nhưng nó được tính lúc đánh giá, nên sáu lượt chạy cũ chưa có số này. Cần chạy lại bước đánh giá trên `predictions.jsonl` để biết dashboard đáng tin đến đâu trước khi đưa cho người dùng thật.
2. **Chạy lại ma trận theo `clean_split`** để có kết quả kèm bảo đảm không rò rỉ cấp bài đánh giá, rồi báo cáo song song hai giao thức. Dữ liệu đã sẵn sàng và đã kiểm tra không rò rỉ; chỉ còn phần huấn luyện, cần máy có VRAM lớn hơn theo Mục 6.5.
3. **Giao thức năm seed** (30 lần huấn luyện, 30 lần đánh giá) để báo cáo trung bình và độ lệch chuẩn.

4. **Sửa bước căn chỉnh XML** của `original_split` cho trường hợp câu trùng lặp.
5. **So sánh theo thời gian trên dashboard:** thêm cột ngày để doanh nghiệp kiểm chứng xem sau khi cải thiện một khía cạnh thì tỷ lệ tiêu cực có giảm thật hay không.
6. **Tăng cường phần phục vụ:** xác thực, giới hạn tần suất và đóng gói container trước khi mở ra ngoài.

Tài liệu tham khảo

[1] Siva Uday Sampreeth Chebolu, Franck Dernoncourt, Nedim Lipka, and Thamar Solorio. OATS: A Challenge Dataset for Opinion Aspect Target Sentiment Joint Detection for Aspect-Based Sentiment Analysis. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 12336–12347, 2024. https://aclanthology.org/2024.lrec-main.1080/.

[2] Mengting Hu, Yike Wu, Hang Gao, Yinhao Bai, and Shiwan Zhao. Improving Aspect Sentiment Quad Prediction via Template-Order Data Augmentation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP 2022)*, pages 7889–7900, 2022. https://aclanthology.org/2022.emnlp-main.538/.

[3] Joseph J. Peper and Lu Wang. Generative Aspect-Based Sentiment Analysis with Contrastive Learning and Expressive Structure. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 6089–6095, 2022. https://aclanthology.org/2022.findings-emnlp.451/.

[4] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[5] Sebastián Ramírez. FastAPI. https://fastapi.tiangolo.com/, 2018.

[6] W3C. Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/, 2023.

[7] Thomas Wolf et al. Transformers: State-of-the-Art Natural Language Processing. https://github.com/huggingface/transformers, 2020.

[8] Wenxuan Zhang, Yang Deng, Xin Li, Yifei Yuan, Lidong Bing, and Wai Lam. Aspect Sentiment Quad Prediction as Paraphrase Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*, pages 9209–9219, 2021. https://aclanthology.org/2021.emnlp-main.726/.

[9] Wenxuan Zhang, Xin Li, Yang Deng, Lidong Bing, and Wai Lam. Towards Generative Aspect-Based Sentiment Analysis. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP 2021), Short Papers*, pages 504–510, 2021. https://aclanthology.org/2021.acl-short.64/.